



ELTE
FACULTY OF
INFORMATICS

PROGRAMMING

Lecture 1

Zsuzsa Pluhár
pluharzs@inf.elte.hu

Introduction



The course

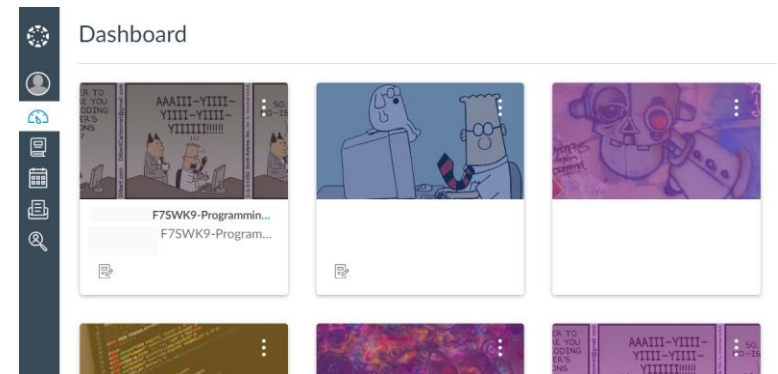
pluharzs@inf.elte.hu
room: 4.724

TOPIC

- basics of programming
- Theory of problem-solving – using patterns
- Programming language (C#) is only a tool to test

CANVAS.ELTE.HU

- tests, assignments
- requirements, RULES !!!
- deadlines



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

Steps of creating a computer program



1. **Specification** (from what?, what?) → specification
2. **Design** (with what?, how?) → data+ algorithmic description
3. **Coding** (how by computer?) → code (representation + implementation)
4. **Testing** (any errors/bugs?) → list of errors (diagnosis)
5. **Searching for bugs** (where is the bug?) → location and reason of bug
6. **Correction** (how would it be correct?) → correct program
7. **Quality assurance, efficiency** (could we make it better? how?) → good program
8. **Documentation** (how does it work?) → usable program
9. **Usage, maintenance** (is it still ok?) → durable program

The specification

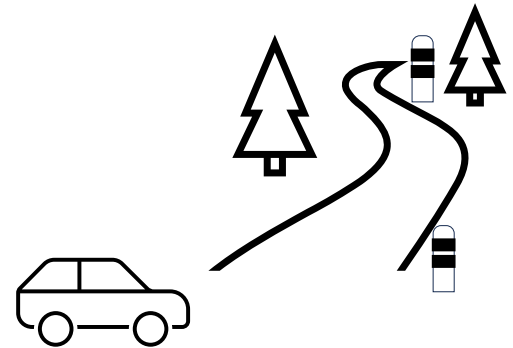


Example task



A family with five children goes on holiday to Lake Balaton. On the motorway, the 11-year-old Sam, who always interested in numbers, can't see the odometer, but wants to know how fast they are going. So, he starts counting how many milestones they have passed and measures the time.

What result does he get?

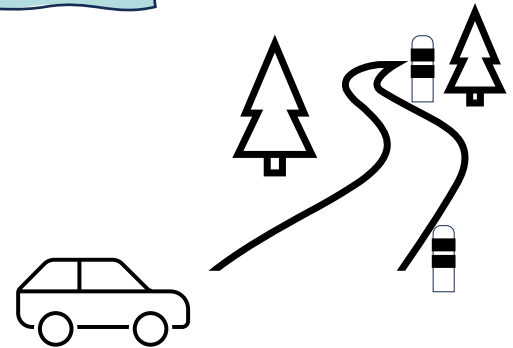


Mark the important information

Example task

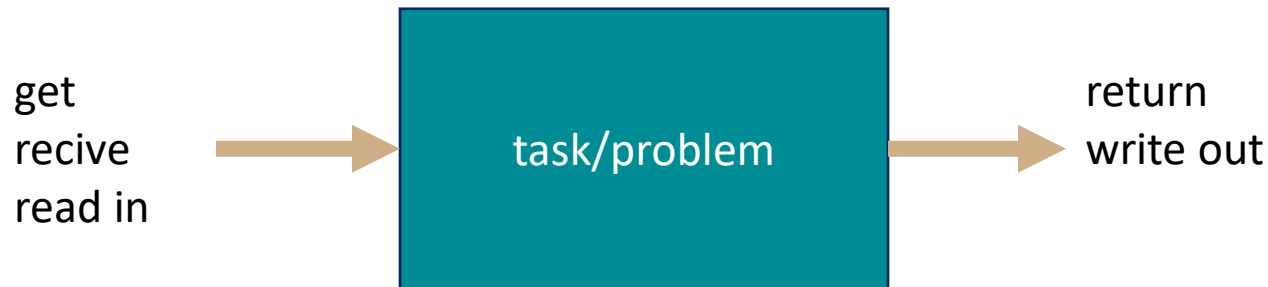
A family with five children goes on holiday to Lake Balaton. On the motorway, the 11-year-old Sam, who always interested in numbers, can't see the odometer, but **wants to know how fast** they are going. So, he starts counting **how many milestones** they have passed and **measures the time.**

What result does he get?



Example task

=> Know the distance and the time and determine the speed.



Example task

Know the distance and the time and determine the speed.

Example values



120km; 1h → 120km/h

12km; 0,1h → 120km/h

10,5km; 0,1h → 105km/h

Data

`s=120; t=1 → v=120`

`s=10,5; t=0,1 → v=105`

- „getting”
 - 2 numbers
 - 2 real numbers
 - 2 real numbers with the „names” (identifiers): s, t
- „return”
 - one number
 - one real number
 - one real number with the „name” (identifier): v

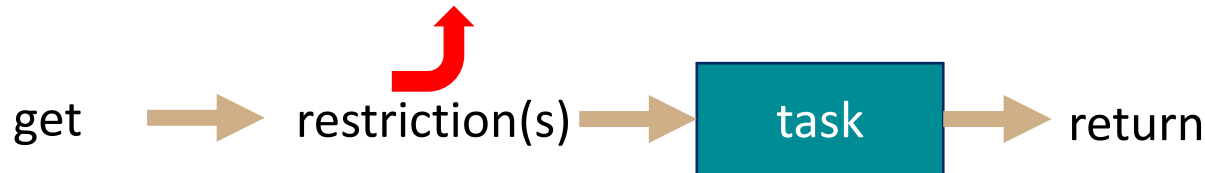
Restriction(s)

Do we accept everything?

$s = -1; \quad t = 0 \rightarrow v = ?$

„getting”

s is a **non-negative** and t is a **positive** real number



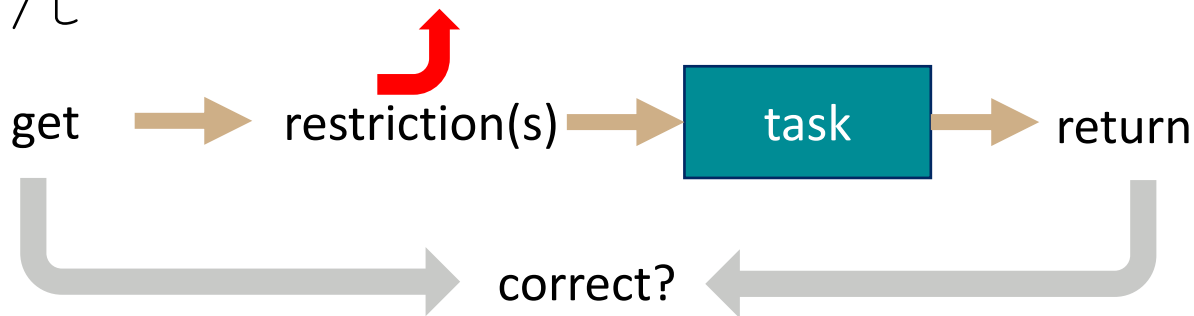
Is the result correct?

How do we know that Sam did the maths correctly?

How do we know that the program **solved the problem correctly**?

Define the relationship between the data given and the data returned!

$$v = s / t$$



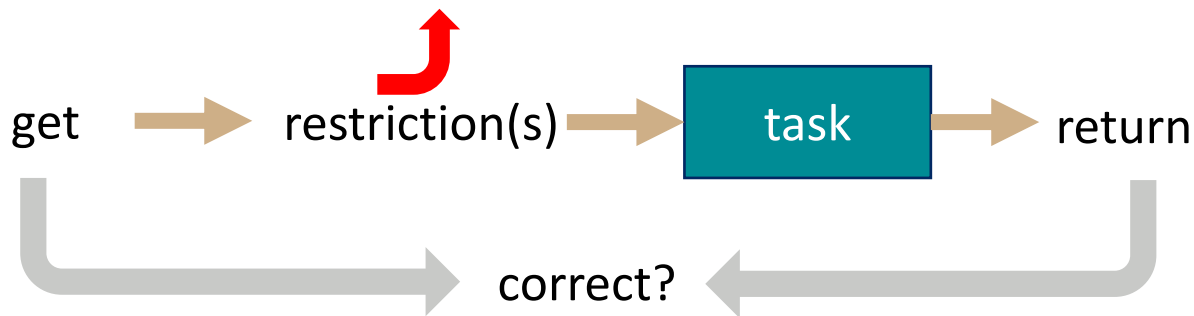
a non-formal description

get: s, t real numbers

return: v real number

restriction: s is non-negative, t is positive

solution: $v = s / t$



Example task – formal description

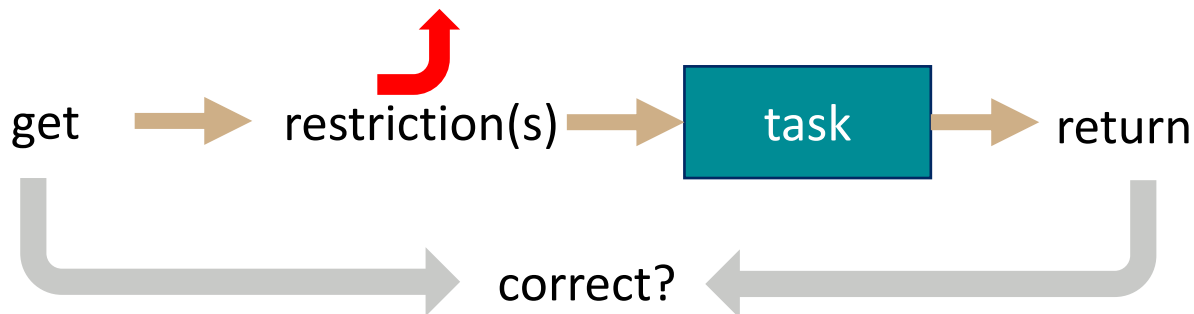
Know the distance and the time and determine the speed.

Input: $s \in \mathbb{R}$, $t \in \mathbb{R}$

Output: $v \in \mathbb{R}$

Precondition: $s \geq 0$ and $t > 0$

Postcondition: $v = s / t$



Example task – formal description

Know the distance and the time and determine the speed.

Specification

Input: $s \in \mathbb{R}$, $t \in \mathbb{R}$

Output: $v \in \mathbb{R}$

Precondition: $s \geq 0$ and $t > 0$

Postcondition: $v = s / t$

$\mathbb{R}$ =set of the real numbers



Specification

Aim: give the task in a formalized way

Questions:

- What does the solution depend on? – input
- What is the solution? – output
- What does it mean to "be the solution"? – postcondition
- Always/When is there a solution? – precondition



Specification



Components:

- **Input data** (identifier, set of values [unit of measure])
- Information about the input (**precondition**)
- **Output**, results (identifier, set of values)
- The statement – true for all input and output – used to get the result (**postcondition**)
- Definitions of the used concepts
- Requirements against the solution
- Restricting conditions



Specification

- **Attributes:**
 - „Unambiguous”, succinct, precise, complete
 - Short, compact, formalized (well-structured)
 - Expressive, understandable (concepts)
- **Tools of specification:**
 - Text description
 - Mathematical formulas

Example task

Know the distance and the time and determine the speed.

Specification

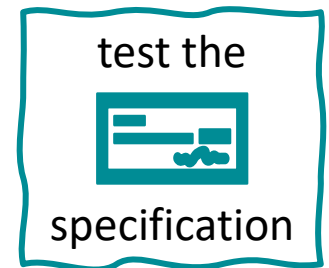
Input: $s \in \mathbb{R}$, $t \in \mathbb{R}$

Output: $v \in \mathbb{R}$

Precondition: $s \geq 0$ and $t > 0$

Postcondition: $v = s / t$

Is the postcondition correct?



The specification as a function

speed: $\mathbb{R} \times \mathbb{R} \rightarrow \mathbb{R}$
speed(s, t)= v and $v=s/t$
speed(s, t)= s/t

- **Input:** $s \in \mathbb{R}, t \in \mathbb{R}$



the domain of the function: $\mathbb{R} \times \mathbb{R} = \mathbb{R}^2$ (the components of which can be referred to in the specification as s, t)

- **Output:** $v \in \mathbb{R}$

the range of the function: $\mathbb{R}$ (which we can be referred in the specification as v)

- **Precondition:** $s \geq 0$ **and** $t > 0$

limiting the domain of the function ($\mathbb{R}^2$) to positive numbers ($\mathbb{R}_{0,+}^2$)

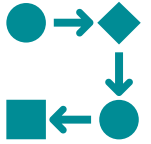
- **Postcondition:** $v = s/t$

what is true of the final result: the „rule” of getting the solution

The algorithm

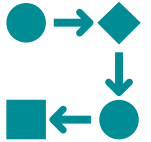


The algorithm



- How can we solve the problem/task?
- The solution broken down into elementary steps:
 - read the input
 - „calculation” of the result
 - write the output

The algorithm



Elements of the algorithms:

- Sequence (step by step execution)
- Conditional branch (choice from 2 or more options based on a condition)
- Loop (repeat certain amount of times or until a certain condition is met)
- Subprogram (a complex activity, with a unique name – abstraction)

Algorithm – the language

- Textual description
 - Writing with sentences
 - Sentence-like elements – pseudocode
- Drawings
 - Flow chart
 - **Structogram**
(Nassi–Shneiderman diagram)

Program QuadraticEquation:

Variables

a,b,c,x:Real

exists:Boolean

d:Real

In: a,b,c [a≠0]

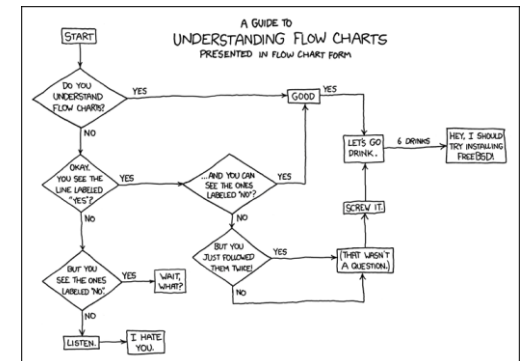
d:=b²-4*a*c

exists:=(d≥0)

If exists **then**

...

End of Program



In: a,b,c [a≠0]	
d:=b ² -4*a*c	
exists:=d≥0	
T	exists?
x:=(-b+√d)/2*a	
	-

Algorithm - pseudocode

sequence

```
Statement1  
Statement2
```

conditional branch

```
If Condition(s) then  
    Statements of true branch  
else  
    Statements of false branch  
End If
```

```
Conditional  
    Condition1 then Statements1  
    Condition2 then Statements2  
    ...  
    else Statements  
End of Conditional
```

loop

```
Loop while Condition  
    loop cycle statements  
Loop end
```

pretesting the condition

```
Loop  
    loop cycle statements  
while Condition  
Loop end
```

posttesting the condition

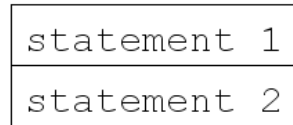
```
Loop from lv=begin to end  
    loop cycle statements  
Loop end
```

counter loop

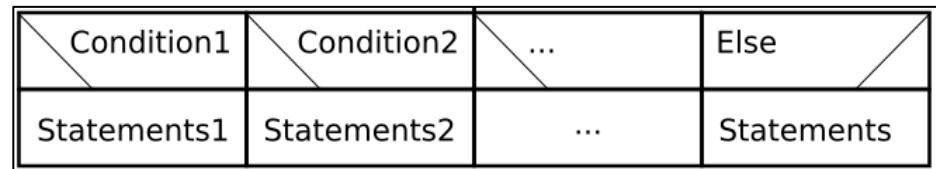
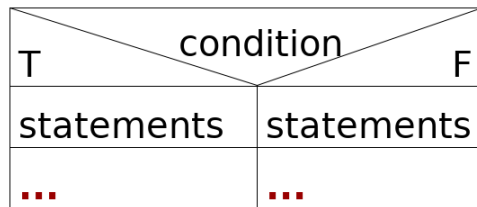


Algorithm - structogram

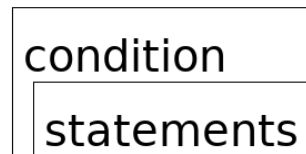
sequence



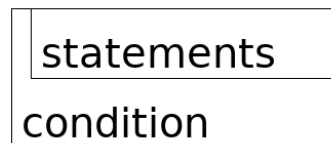
conditional branch



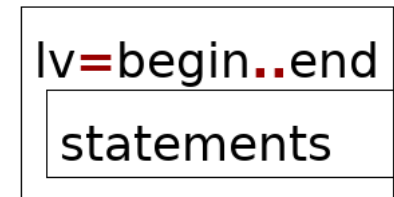
loop



pretesting the condition



posttesting the condition



counter loop

Algorithm - structogram

creating/drawing algorithm

<https://progalap.elte.hu/stuki/>

spreadsheet/word editor

[Structorizer](#)

[CodeBlocks](#) NSD tool

[Online](#) editor (special language)



Specification and implementation

Specification:

→ objects in the real world – their representation in the real world (e.g. the set of real numbers)

→ „give” them to the computer – store them into variables (in memory) – implement only a part of the original set – **type**

- Next steps:

- Calculate the result with the help of functions and store it into memory
- Give back the result into the real world



Specification and implementation

- Some variables are used only in calculating.
- A specification of a method can talk about the parameters and return value of the method, but it should **never talk about local variables of the method** or private fields of the method's class.
- You should consider the implementation invisible to the reader of the specification.

Specification and algorithm

- Specification:
 - assign data to data: data $\rightarrow$ data
 - data can be referred by identifiers („names”, labels)
- Algorithm:
 - we can use the same identifiers for variables
 - Before the algorithm is executed, the input variables take the values of the input data (read the input)
 - The algorithm can modify the values of the variables during the execution (processing, solving)
 - After the algorithm has been executed, the output variables must contain the values of the output data (write-out)



Example: speed

Algorithm

The program's 4 sections: **declaring the data**; **reading the input**; **calculating** the result; **write the output**

Specification

Input: $s \in \mathbb{R}, t \in \mathbb{R}$

Output: $v \in \mathbb{R}$

Precondition: $s \geq 0$ and $t > 0$

Postcondition: $v = s / t$

Variable

$s, t: \text{Real}$

$v: \text{Real}$

In: s, t $\rightarrow [s \geq 0 \text{ and } t > 0]$

$v := s / t$

Out: v



Example: speed

$v := s / t$

Algorithm

- don't write the algorithm of „reading input”, „writing output”
- Don't write the clearly derivable declarations

Specification

Input: $s \in \mathbb{R}, t \in \mathbb{R}$

Output: $v \in \mathbb{R}$

Precondition: $s \geq 0$ and $t > 0$

Postcondition: $v = s / t$

~~In: s, t [$s \geq 0$ and $t > 0$]~~

$v := s / t$

~~Out: v~~

~~Variable
 $s, t: \text{Real}$
 $v: \text{Real}$~~

The code





code – the frame

```
namespace speed {  
    internal class Program {  
        static void Main(string[] args) {  
            // declaration  
  
            // reading the input  
  
            // "processing", solving the task  
  
            // writing the output  
  
        }  
    }  
}
```

code



```
// declaration
double s, t;
double v;
// reading the input
Console.Write("s = ");
double.TryParse(Console.ReadLine(), out s);

Console.Write("t = ");
double.TryParse(Console.ReadLine(), out t);

// "processing", solving the task
v = s / t;

// writing the output
Console.WriteLine("The speed: {0}", v);
```



code



```
// declaration and reading input
// ...
// checking precondition
if (s >= 0 && t > 0) {
    // "processing"
    v = s / t;

    // writing the output
    Console.WriteLine("The speed: {0}", v);
}
else {
    Console.WriteLine("No accessible value!
Run it again!");
}
```

code - declaration

specification	algorithm	code
$\mathbb{Z}$	Integer	sbyte, short, int , long
$\mathbb{N}$	Natural	byte, ushort, uint , ulong
$\mathbb{R}$	Real	float, double
$\mathbb{L}$	Logical	bool
$\mathbb{S}$	String	string
$\mathbb{C}$	Char	char

$s \in \mathbb{R} \rightarrow s: \text{Real} \rightarrow \text{double } s_i$

code: reading the input, writing the output



- Reading the input

```
Console.Write("s = ");  
double.TryParse(Console.ReadLine(), out s);
```

- Writing the output

```
Console.WriteLine("The speed: {0}", v);  
Console.WriteLine($"The speed: {v}");
```

- Processing, calculating the result

- value assignment: = operator
 $v = s / t;$

Conditional statement



Triangle

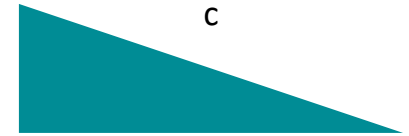


Could the given 3 numbers be the sides of a right-angled triangle?

Solution:

- yes, if $a^2 + b^2 = c^2$
- no, if the $a^2 + b^2 = c^2$ not met

Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Specification:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $can \in \mathbb{L}$

Precondition: $a > 0$ and $b > 0$ and $c > 0$

Postcondition:

$(a^2 + b^2 = c^2 \text{ and } can = \text{true}) \text{ or } (a^2 + b^2 \neq c^2 \text{ and } can = \text{false})$

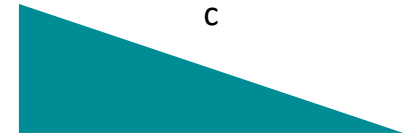
test the



specification with:

a: 3
b: 4
c: 5.00000000000001
can: false

Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Specification:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $can \in \mathbb{L}$

Precondition: $a > 0$ and $b > 0$ and $c > 0$

Postcondition:

$(a^2 + b^2 = c^2 \text{ and } can = \text{true}) \text{ or } (a^2 + b^2 \neq c^2 \text{ and } can = \text{false})$

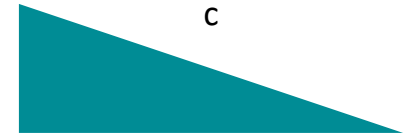
the order of the sides is defined **implicite**

modify the



specification

Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Specification₂:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $can \in \mathbb{L}$

Precondition: $a > 0$ and $b \geq a$ and $c > b$

Postcondition:

$(a^2 + b^2 = c^2 \text{ and } can = \text{true}) \text{ or }$

$(a^2 + b^2 \neq c^2 \text{ and } can = \text{false})$

the order of the sides is defined **explicite**

Triangle



Implication:

the logical expression of the "if...else,,

Postcondition:

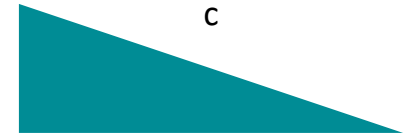
$(a*a+b*b=c*c \text{ and } \text{can}=\text{true}) \text{ or } (a*a+b*b \neq c*c \text{ and } \text{can}=\text{false})$

Postcondition:

$(a*a+b*b=c*c \rightarrow \text{can}=\text{true}) \text{ and } (a*a+b*b \neq c*c \rightarrow \text{can}=\text{false})$

a	b	a->b
true	true	true
true	false	false
false	true	true
false	false	true

Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Specification:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $can \in \mathbb{L}$

Precondition: $a > 0$ and $b \geq a$ and $c > b$

Postcondition:

$(a^2 + b^2 = c^2 \text{ and } can = \text{true})$ or

$(a^2 + b^2 \neq c^2 \text{ and } can = \text{false})$

There can be problems with real-type equality due to the representation of real-type data.
The exact solution is:

$$|a^2 + b^2 - c^2| < \text{epsilon}$$

Algorithm

T	$(a^2 + b^2 = c^2)$		F
can := true		can := false	

Triangle - code

There can be problems with real-type equality due to the representation of real-type data. The exact solution is:

$$|a^2 + b^2 - c^2| < \text{epsilon}$$

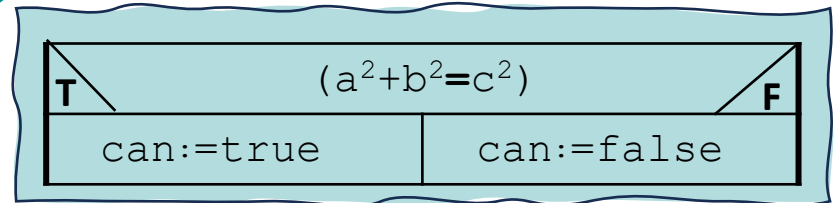
```
// declaration
double a, b, c;
bool can;
// reading the input
Console.Write("a = ");
double.TryParse(Console.ReadLine(), out a);

Console.Write("b = ");
double.TryParse(Console.ReadLine(), out b);

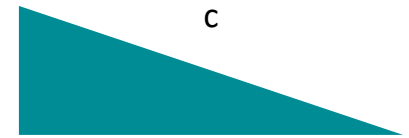
Console.Write("c = ");
double.TryParse(Console.ReadLine(), out c);

// solution
if (a * a + b * b == c * c) {
    can = true;
}
else {
    can = false;
}

// writing the output
Console.WriteLine("It is possible: {0}", can);
```



Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Specification₃:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $can \in \mathbb{L}$

Precondition: $a > 0$ and $b \geq a$ and $c > b$

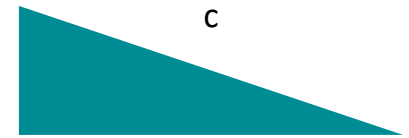
Postcondition: $can = (a^2 + b^2 = c^2)$

Algorithm:

can := ($a^2 + b^2 = c^2$)



Triangle



Could the given 3 numbers be the sides of a right-angled triangle?

Algorithm – using helper variable:

$aa := a^2$
$bb := b^2$
$cc := c^2$
$can := (aa + bb = cc)$

Variable:
 $aa, bb, cc : \text{Real}$

extra/helper variable

Don't written in the specification!

$$ax^2+bx+c=0$$

Example: quadratic equation

Let's specify **one root** of a quadratic equation! The equation is: $ax^2+bx+c=0$

Specification₁:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $x \in \mathbb{R}$

Precondition: -

Postcondition: $ax^2+bx+c=0$

Comment: the postcondition does not provide us with information how to create the algorithm. No problem, it's usually the case, but let's try again...

Formula of solution:

$$x_{1,2} = \frac{-b \pm \sqrt{b^2 - 4ac}}{2a}$$



$$ax^2+bx+c=0$$

Example: quadratic equation

Let's specify **one root** of a quadratic equation! The equation is:
 $ax^2+bx+c=0$

Specification₂:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $x \in \mathbb{R}$

Precondition: $a \neq 0$

Postcondition: $x = (-b + \sqrt{b*b - 4*a*c}) / (2*a)$

Questions:

Is there always a solution?/When is there a solution?

Are we sure there is only one solution?



$$ax^2+bx+c=0$$

Example: quadratic equation

Let's specify **one root** of a quadratic equation! ...

Specification₃:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $x \in \mathbb{R}$, **exists** $\in \mathbb{L}$

Precondition: $a \neq 0$

Postcondition: **exists** $= (b*b - 4*a*c \geq 0)$ and
exists $\rightarrow x = (-b + \text{sqrt}(b*b - 4*a*c)) / (2*a)$

Are we sure there is only one solution?

complete the



specification



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

$$ax^2+bx+c=0$$

Example: quadratic equation

Let's specify **one root** of a quadratic equation! ...

Algorithm:

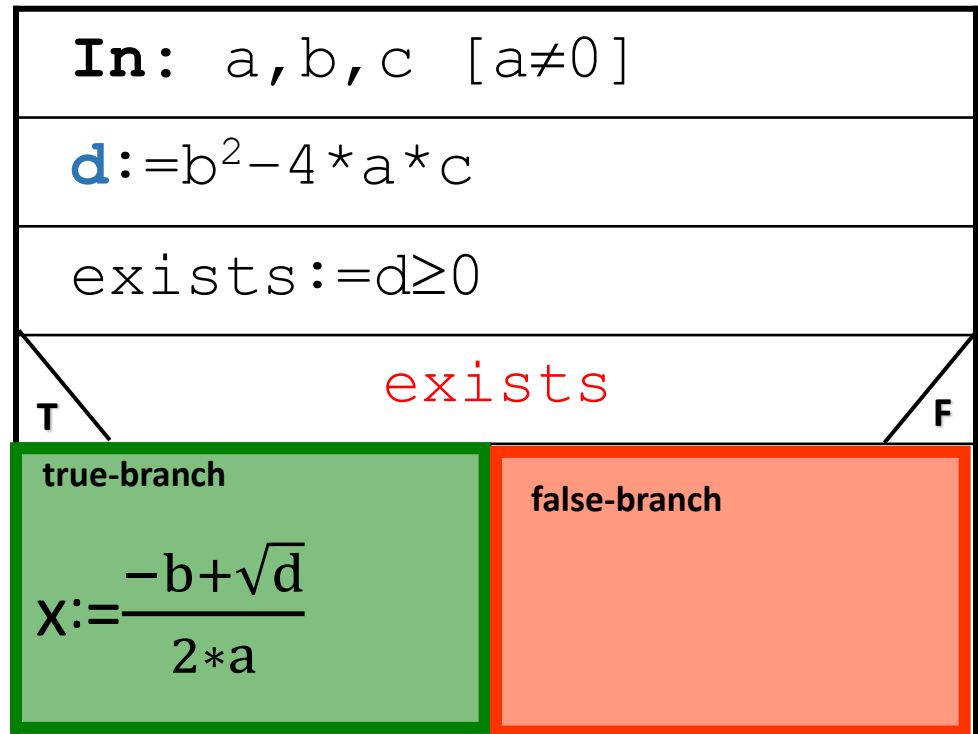
Specification₃:

Input: $a \in \mathbb{R}$, $b \in \mathbb{R}$, $c \in \mathbb{R}$

Output: $x \in \mathbb{R}$, $\text{exists} \in \mathbb{L}$

Prec.: $a \neq 0$

Postc.: $\text{exists} = (b*b - 4*a*c \geq 0)$ and
 $\text{exists} \rightarrow x = (-b + \sqrt{b*b - 4*a*c}) / (2*a)$



Concepts related to data



Concepts related to data

Constant

- Data that **cannot change** its **value** during statement execution. It preserves its *state*.

Variable

- As the name indicates, its actual value may change, so statements can assign a new value to the variable. In a more scientific phrasing: the state set of the variable has more elements.

Concepts related to data

Identifier

- A sequence of symbols that allows us to refer to the data content, as well as to modify the data content.

Initial value

- The value assigned at the “birth” of the identifier.
- In the case of constants, this is an explicit value, in the case of variables this depends on how the actual programming environment handles the initial values.



Concepts related to data

Value assignment ($:=$)

- A statement that changes the value of the identifier. Thus the identifier gets from its actual state to a new state. (We cannot assign new values to a constant.)

Type

- This is a “contract” for a variable (or constant): we define what kind of data the variable may represent, so that we can have a fixed set of states, and applicable operations.

Type

From the perspective of complexity (structuredness) we differentiate between

- **unstructured** (scalar, elementary) data type: it cannot be broken into pieces (at least on this observation level)
- **structured** (compound) data type: formed using elementary types

Elementary types: integer

4-byte representation

- **Value set:** $-2^{31} \dots +2^{31} - 1$
(Min'Integer..Max'Integer)
- **Operators:** $+$, $-$, $*$, div (integer division), mod (remainder), $-$ (unary minus), $^$ (power to positive exponent)
- **Relational operators:** $=$, $<$, $\neq$, $\leq$, $\geq$, $>$
- **Representation:** two's complement
- **Variations:** depends on sign and size

Beyond
reading in,
writing out
and value
assignment



Eg. 3-bit two's complement integers:

$+0=0|00_2$, $+1=0|01_2$, $+2=0|10_2$, $+3=1|11_2$,
 $-1=1|11_2$, $-2=1|10_2$, $-3=1|01_2$, $-4=1|00_2$

Can you figure out the rule?



ELTE

FACULTY OF
INFORMATICS

PROGRAMMING – Zsuzsa Pluhár

Elementary types: real

- **Value set:** ??? ... ??? (Min'Real..Max'Real are not defined, or depend on implementation)
- **Operators:** $+$, $-$, $*$, $/$, $^$, $-$ (unary minus)
- **Relational operators:** $=$, $<$, $\neq$, $\leq$, $\geq$, $>$
- **Representation:** floating point (could be more precisely called „rational number” as it can only represent those numbers)

Floating point number

• Decimal: -3.123×10^{-11}

Sign Mantissa Base/Radix Exponent

• Mantissa = $-(1 \times 10^{-1} + 2 \times 10^{-2} + 3 \times 10^{-3})$

• Binary: -1.1101×2^{110}

• Mantissa = $-(1 \times 2^0 + 1 \times 2^{-1} + 1 \times 2^{-2} + 0 \times 2^{-3} + 1 \times 2^{-4}) = -1.8125$

• Exponent = $(1 \times 2^2 + 1 \times 2^1 + 0 \times 2^0) = 6$

Elementary types: character type

- **Value set:** $0 \dots 255$ – coded sign, ASCII
(Min'Character..Max'Character: the 0 and the 255)
- **Operators:** character-specific (only int)
 - (maybe function $\text{Code} : \text{Character} \rightarrow \text{Integer}$, and its inverse, the function $\text{Character} : \text{Integer} \rightarrow \text{Character}$, but we use these only in relation to internal representation)
- **Relational operators:** $=, <, \neq, \leq, \geq, >$
(based on their internal representation $\rightarrow$ not alphabetical order!)

Elementary types: Boolean (logical) type

- **Value set:** `False..True` – coded sign, ASCII
(Min' Boolean..Max' Boolean: the False and the True)
- **Operators:** `not`, `and`, `or` (usual logical operators)
- **Relational operators:** `=`, `<`, `≠`, `≤`, `≥`, `>`
- **Representation:** `0=False`, `1=True` (or any non-0 value=True)
- **Comment:** sorting does not really have relevance

